

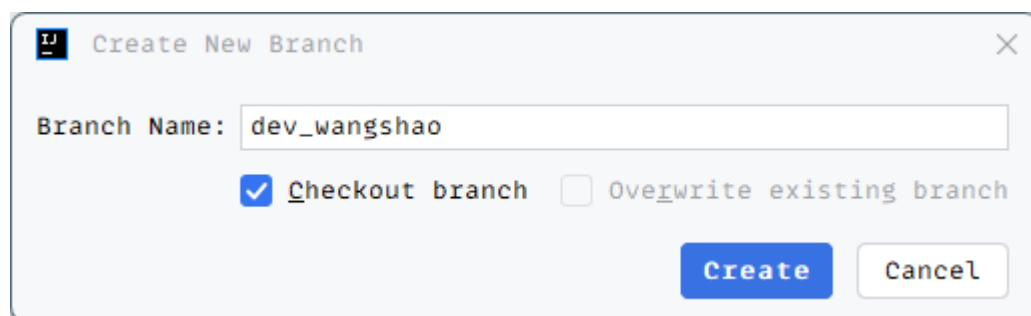
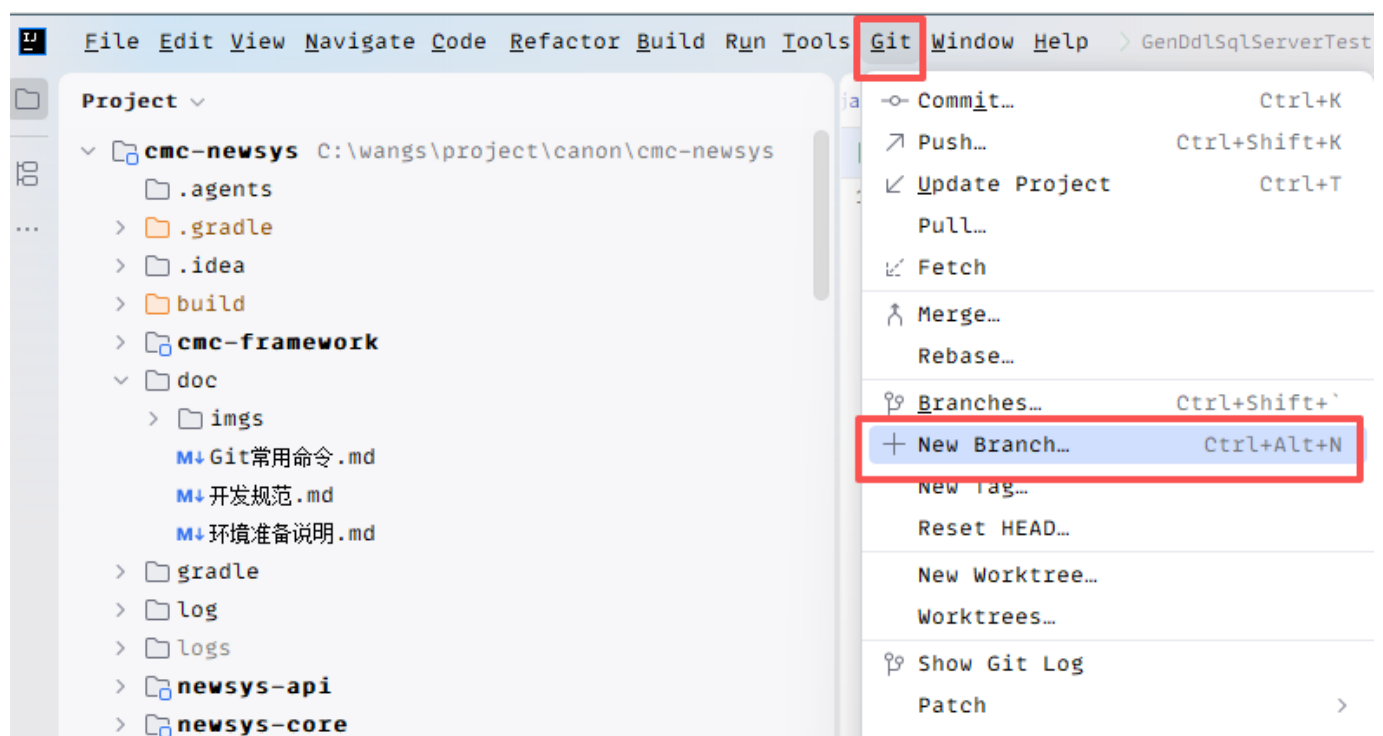
全栈开发入门示例

本示例是以全新的功能模块为例，所以过程中会涉及到模块和路由等基础配置。

后端开发

拉取新分支

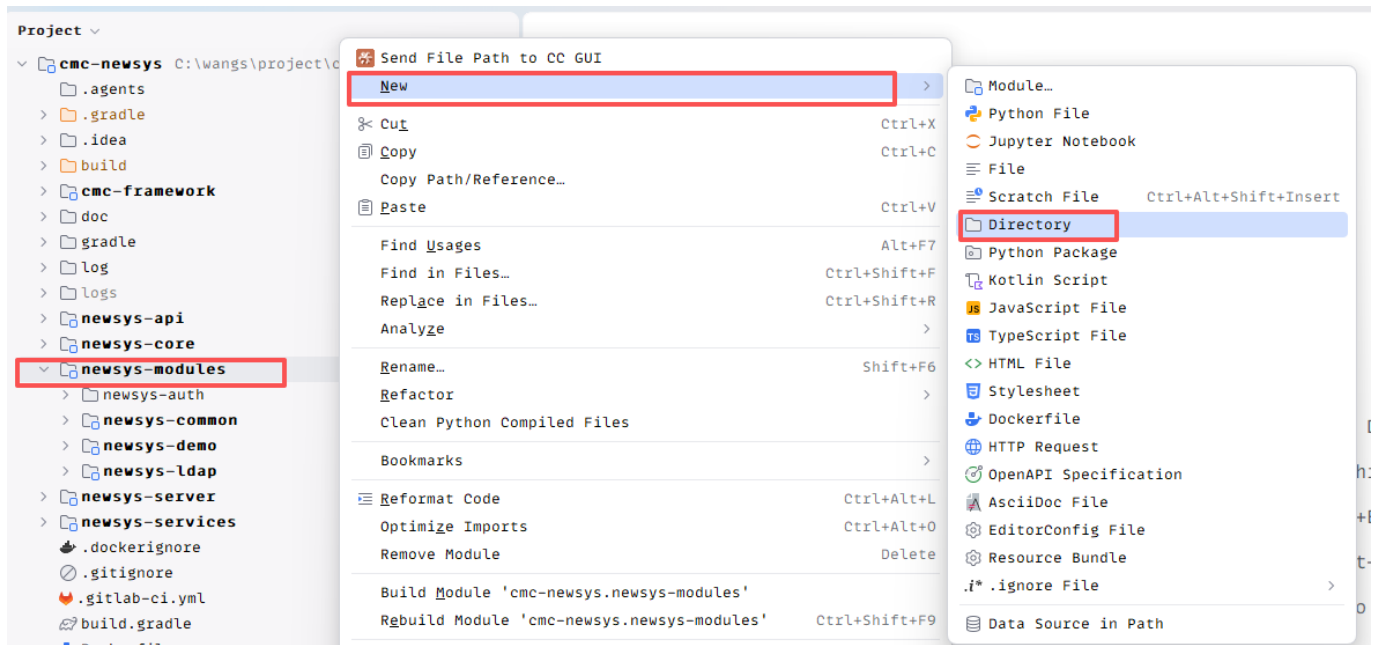
为避免影响已有功能及开发人员的测试功能冲突，建议大家拉取以自己名称命名的新分支，操作：在IDEA的工具栏中【Git】—>【New Branch】



新建模块

在功能模块文件夹 newsys-modules 下新建目录newsys-test,

操作：右键【newsys-modules】—>【New】—>【Directory】



输入完目录名称后点击回车即可，然后在该目录下创建Gradle的构建文件 `build.gradle`，文件内容如下

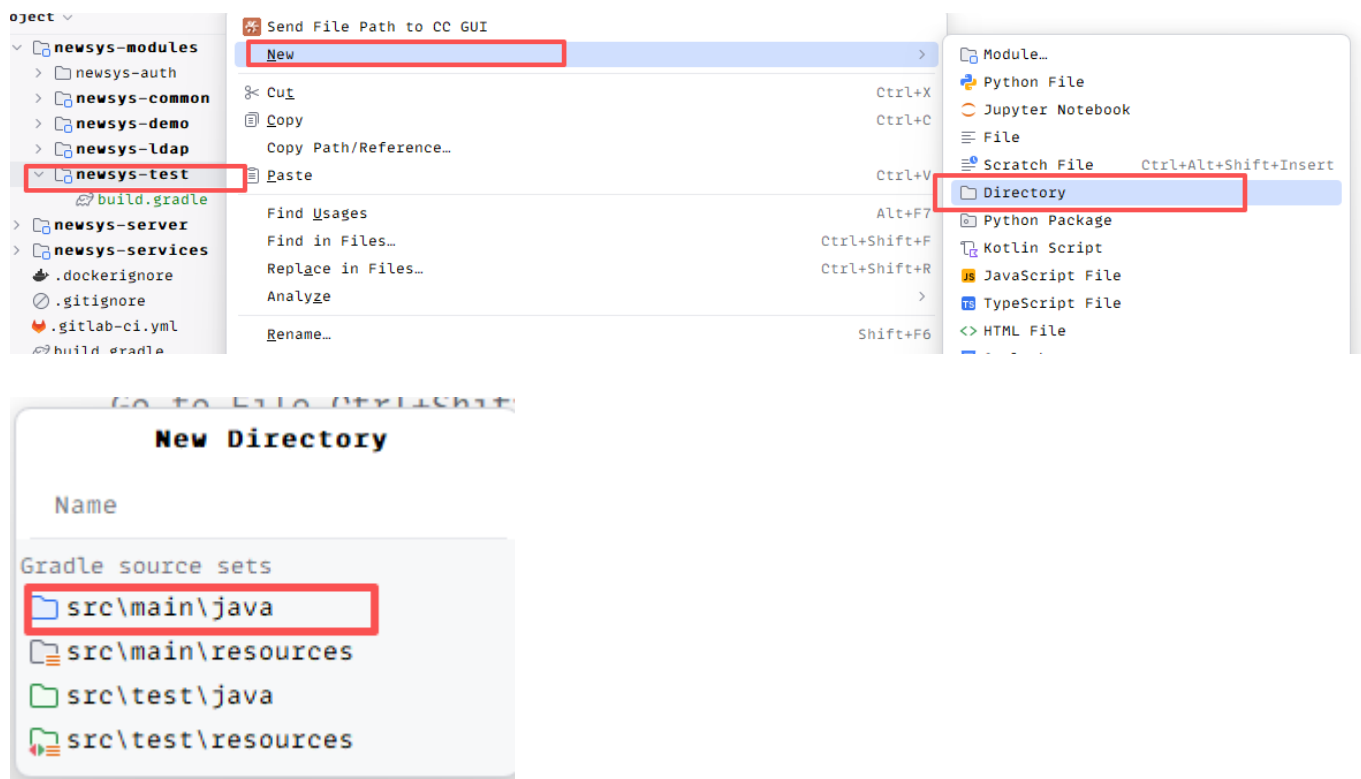
```
dependencies {  
    api project(':newsys-modules:newsys-common')  
}
```

创建完成后，刷新IDEA右侧的Gradle，



待刷新完成后（注意：刷新成功后新建的目录名变黑，说明模块添加成功），右键点击新加的模块进行添加源码目录，如

操作：右键【newsys-test】—>【New】—>【Directory】—>选择【src/main/java】后双击

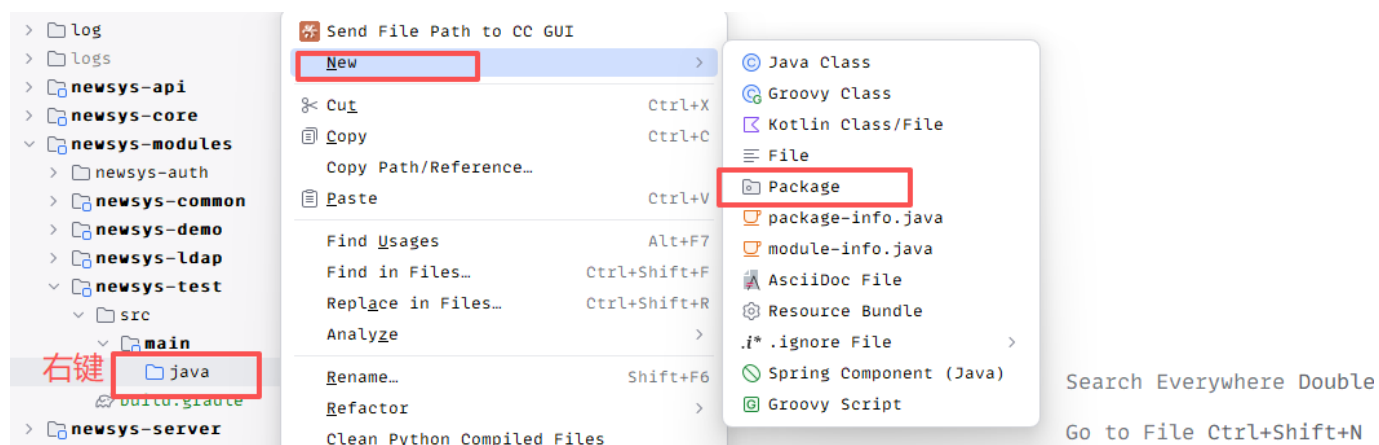


双击后就会在新的模块下自动创建出src/main/java目录。

提供服务接口

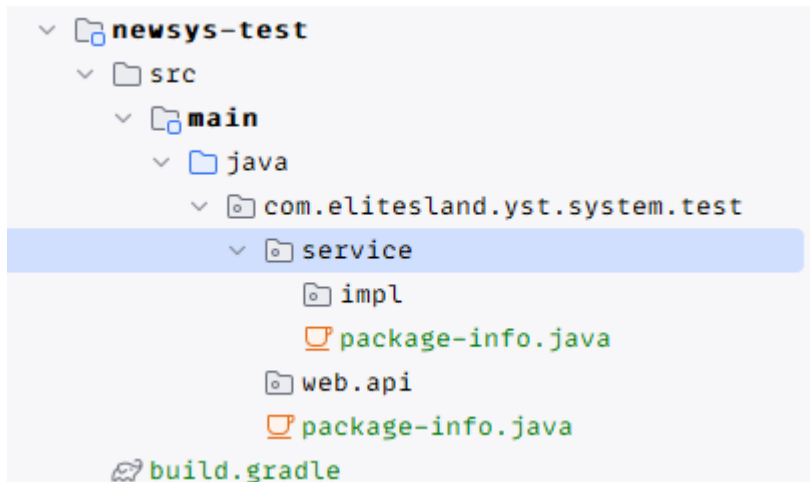
新建包

操作：右键新创建的java目录【java】—>【New】—>【Package】



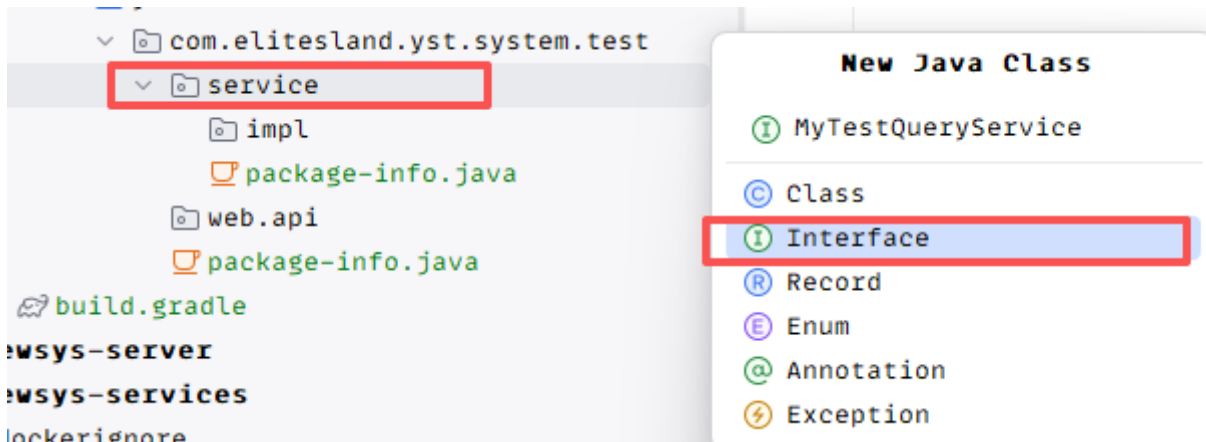
输入包路径 `com.elitesland.yst.system.test` 后回车即可，可以在其下面创建个package-info.java占位。操作：右键新创建的包【com.elitesland.yst.system.test】—>【New】—>【package-info.java】

然后在该目录下分别创建以下子包：service、service.impl、web.api，最终结果

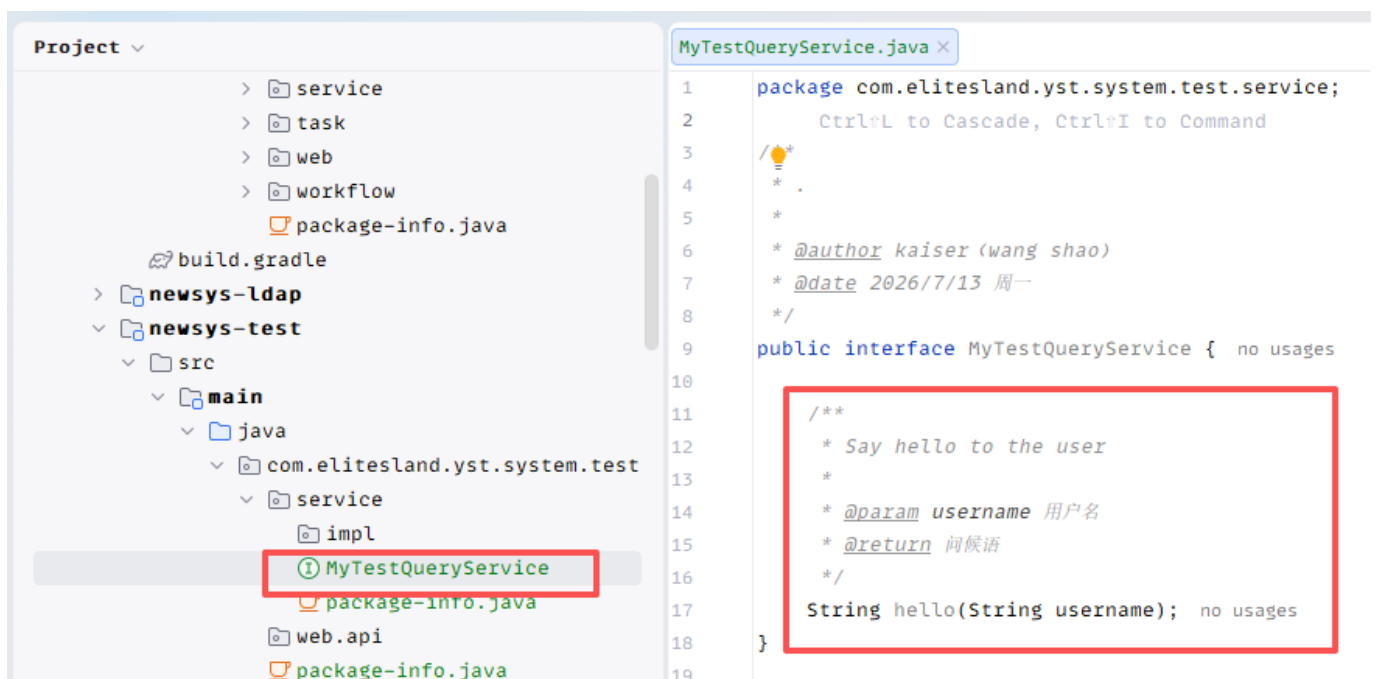


创建查询服务接口

操作：右键【service】包—>输入接口类名【MyTestQueryService】—>双击【Interface】



在接口中定义个简单的方法，包含出参和入参



实现服务查询接口

操作：右键【impl】包—>输入接口实现类名【MyTestQueryServiceImpl】—>双击【Class】

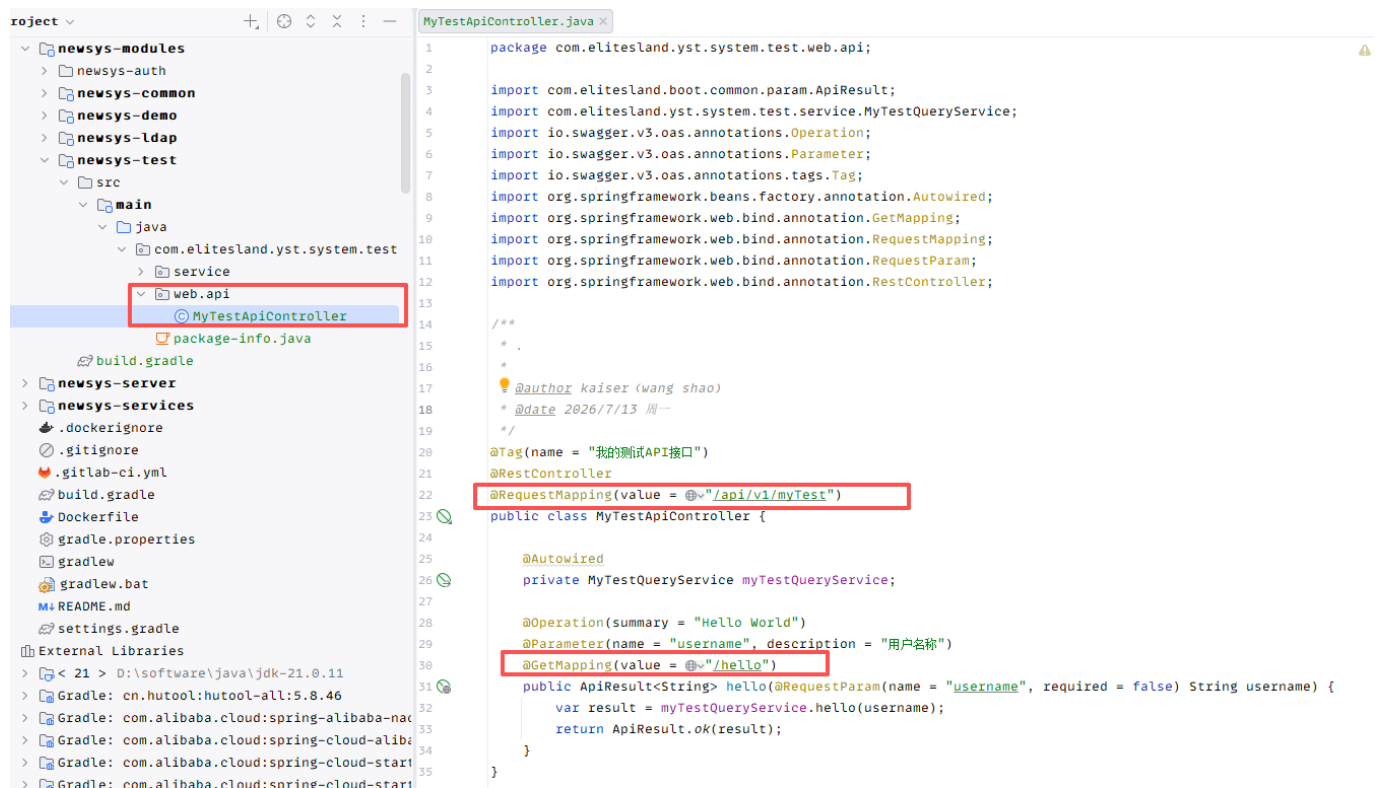
在实现类中需要实现上面新创建的服务接口MyTestQueryService（即通过关键字implements）



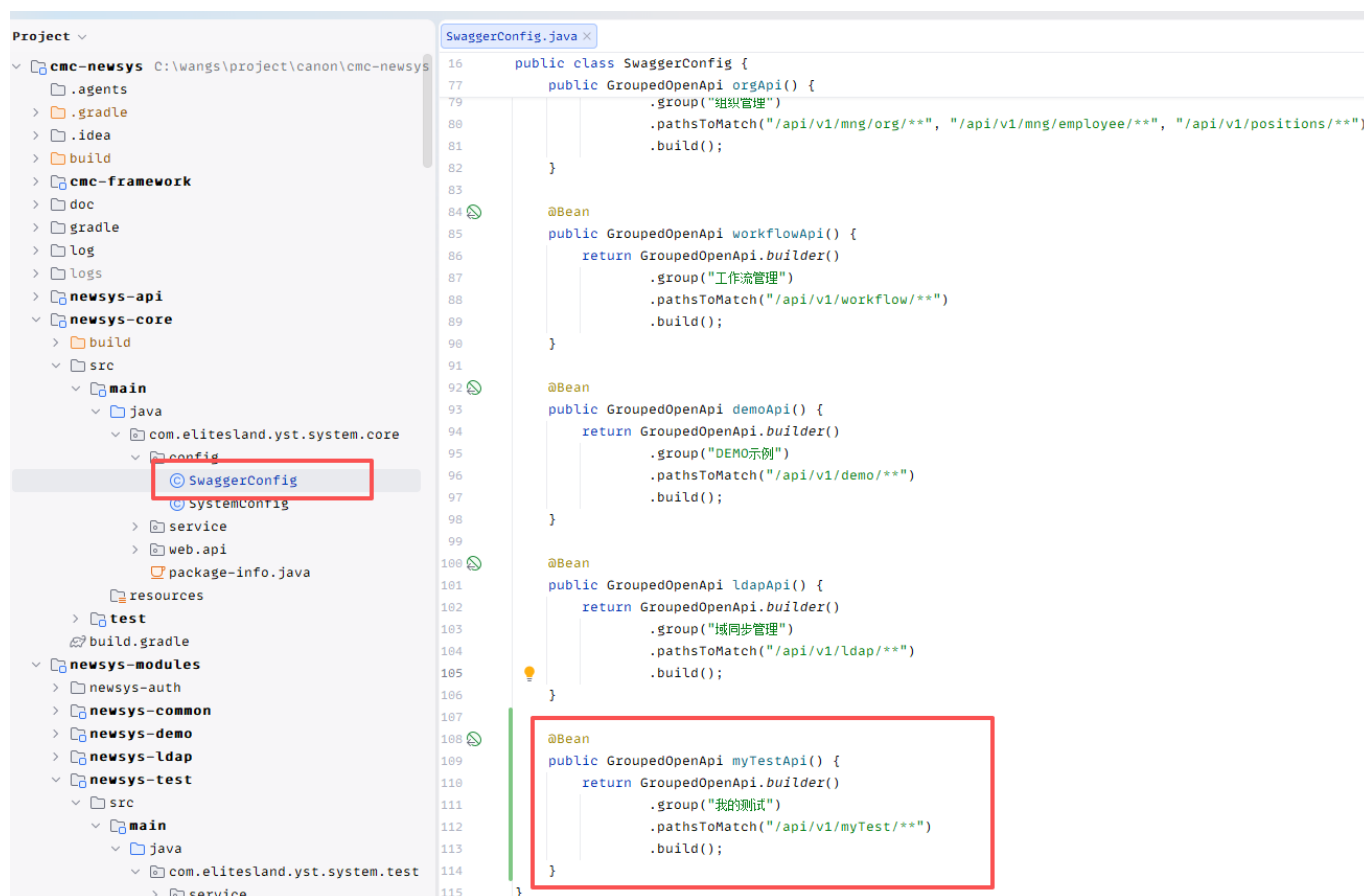
创建API接口

提供API接口供前端调用

操作：右键【api】包—>输入控制器类名【MyTestApiController】—>双击【Class】



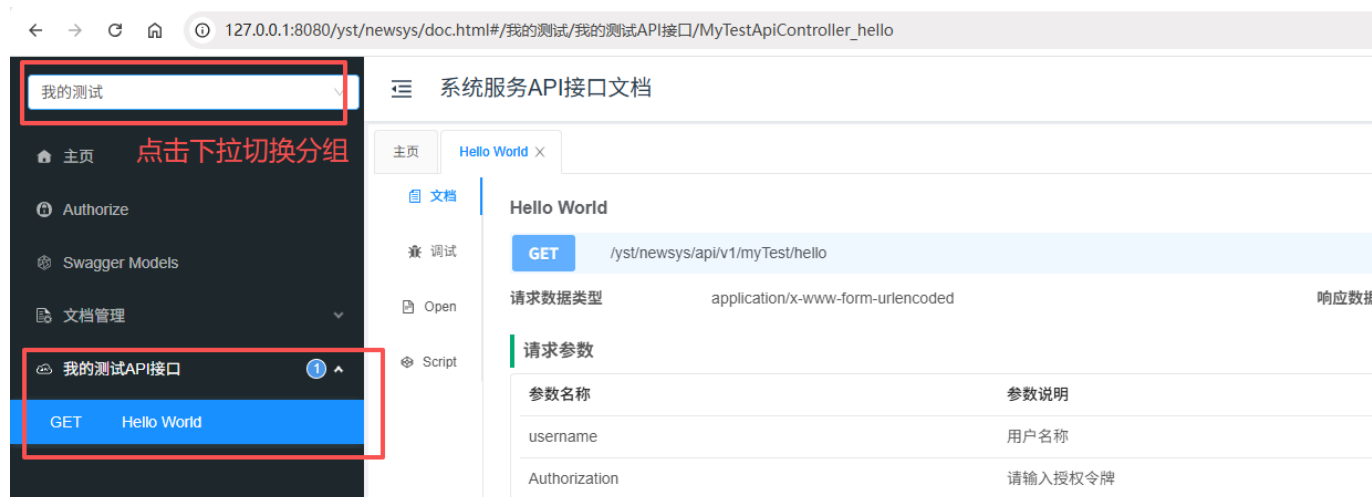
定义API接口，类上的@RequestMapping中的value的值和方法上的@GetMapping中的value的值拼接在一起作为这个接口的访问路径。



最后在swagger中加入该控制器的接口路径，启动项目即可看到新加的接口。

访问swagger地址：<http://127.0.0.1:8080/yst/newsys/doc.html>

切换到对应的分组下：我的测试



前端开发

注意：开发前请看下项目根目录下的pageReadme.md文档了解目录、文件等目录和命名规范

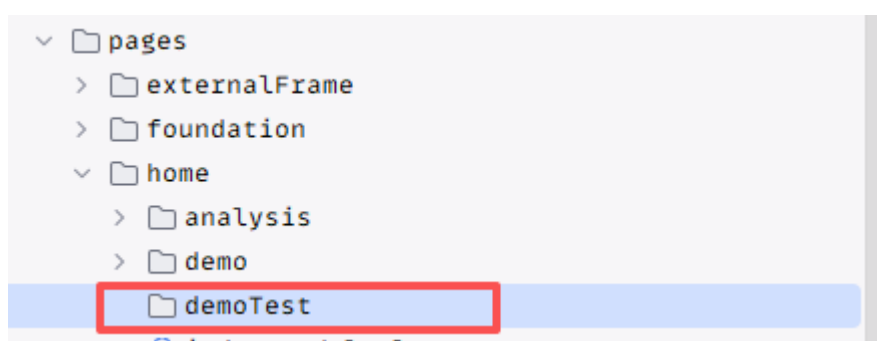
拉取新分支

可参考上面后端的拉取新分支方式。

创建页面目录

注意：pages目录下的一级目录对应我们主页中横向的一级菜单，比如home就是指上方的【首页】，一级目录下的二级目录对应我们左侧的二级菜单，比如demo就是指左侧的【示例页】。

在pages/home目录下创建demoTest目录。



创建首页

在demoTest目录下创建功能的入口页面index.tsx，如

```

import React, { useState } from "react";
import { Button, Input, Space, message } from "antd";

const DemoTestPage: React.FC = () => {
  const [username, setUsername] = useState("");
  const [result, setResult] = useState("");
  const [loading, setLoading] = useState(false);

  // 提交按钮的回调
  const handleSubmit = async () => {
    if (!username.trim()) {
      message.warning("请输入用户名");
      return;
    }

    // 加载loading
    setLoading(true);

    console.log("用户输入: ", username);

    // 取消loading
    setLoading(false);
  };

  // 页面组件的渲染
  return (
    <div style={{ padding: 16 }}>
      <Space orientation="vertical" size={16} style={{ width: 420 }}>
        <Space>
          <span>用户名</span>
          <Input
            value={username}
            onChange={(event) => setUsername(event.target.value)}
            onPressEnter={handleSubmit}
            placeholder="请输入用户名"
            style={{ width: 240 }}
          />
          <Button type="primary" loading={loading} onClick={handleSubmit}>
            调用接口
          </Button>
        </Space>

        <Space align="start">
          <span style={{ lineHeight: "32px" }}>接口返回结果</span>
          <Input.TextArea
            value={result}
            readOnly
            autoSize={{ minRows: 4, maxRows: 8 }}
            style={{ width: 300 }}
          />
        </Space>
      </Space>
    </div>
  );
};

```



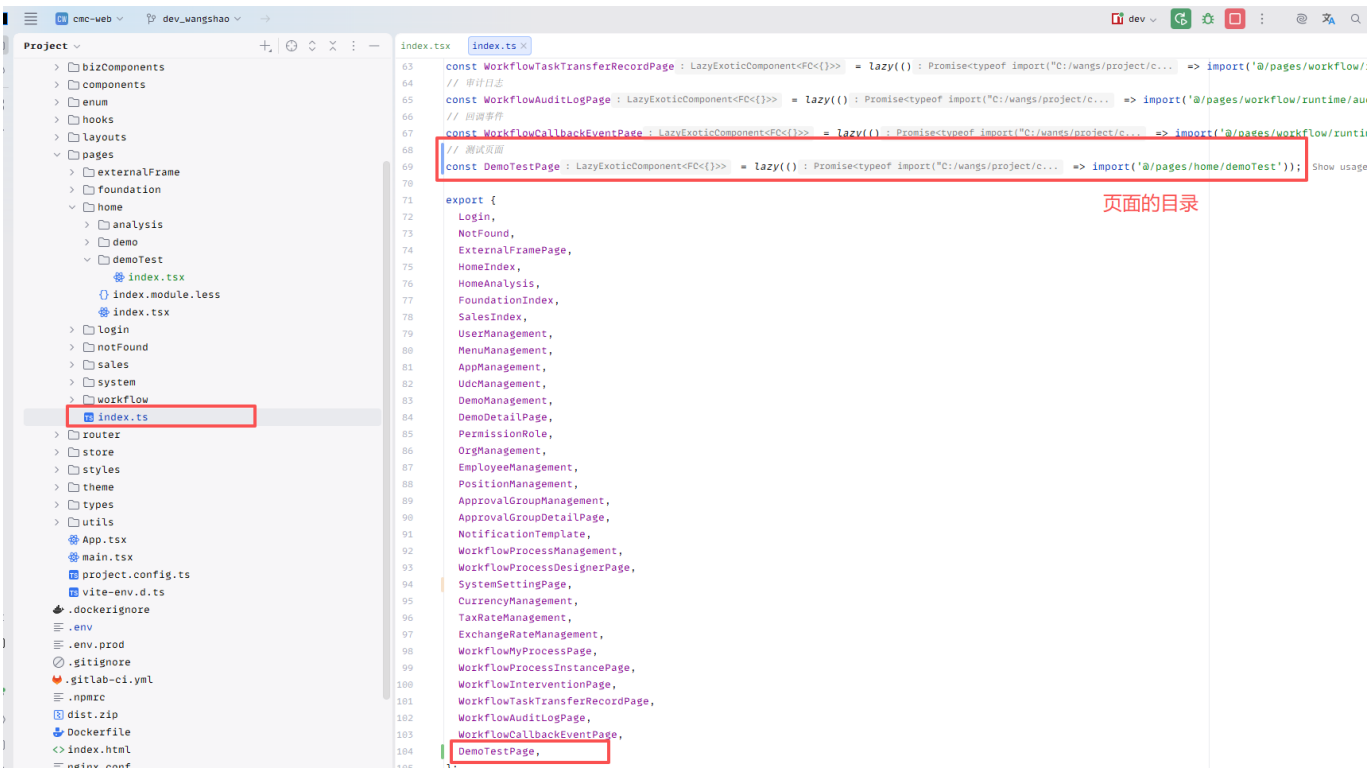
```

        </Space>
      </Space>
    </div>
  );
};

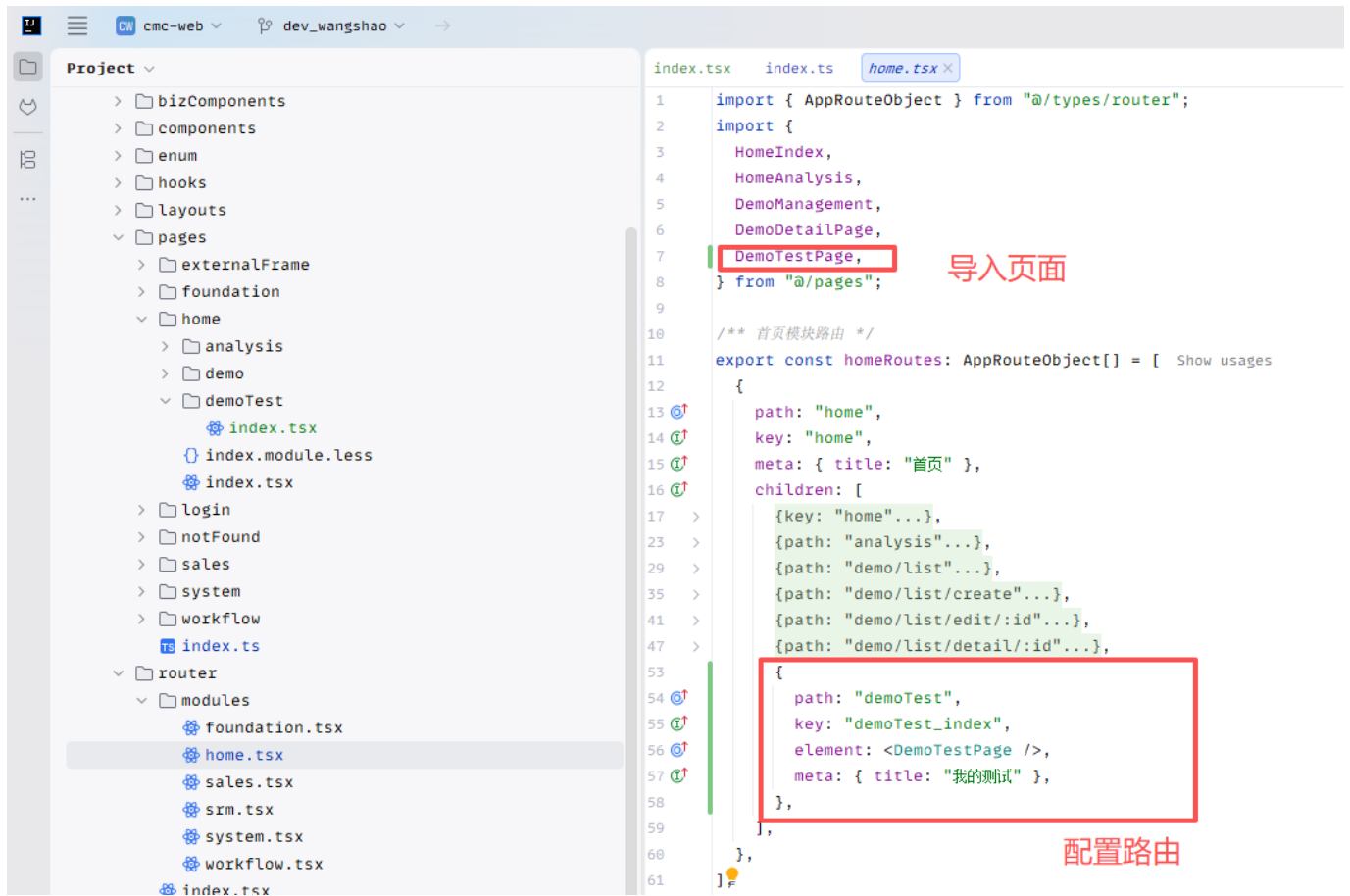
export default DemoTestPage;
```

导出页面

导出页面供路由引用



配置路由



启动前端服务

启动命令：pnpm run dev:local

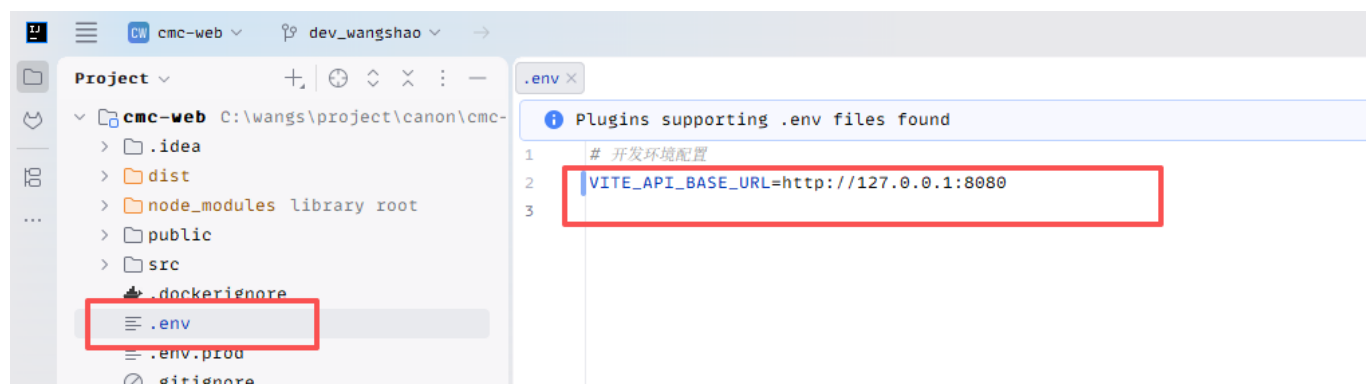
访问<http://127.0.0.1:3000>即可在首页中看到【我的测试】



在文本框中输入内容后点击【调用接口】模拟调用，即可在浏览器控制台（按F12调出控制台）看到输出内容。

调用后端API接口

在.env文件中把后端服务地址改成本地的

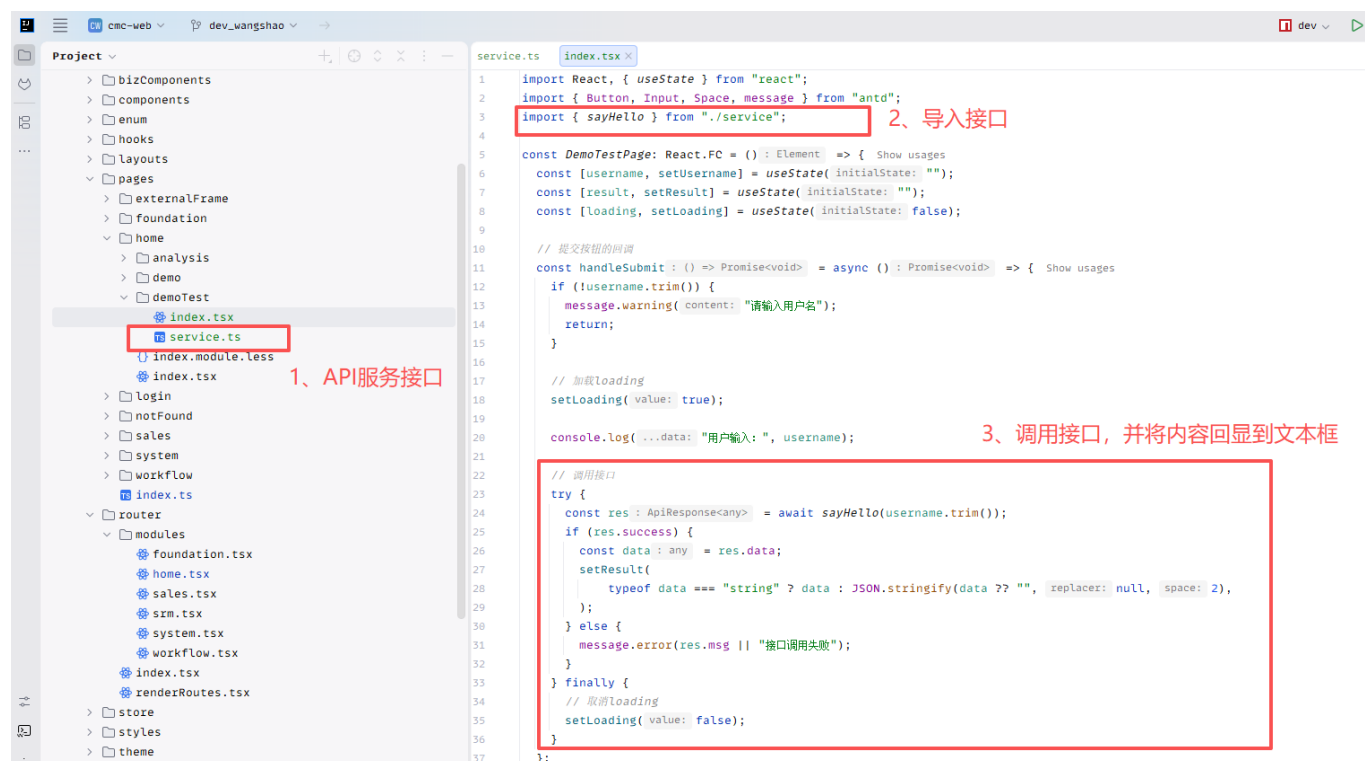


在我们的demoTest目录下添加服务接口配置service.ts文件，文件内容

```
import request from "@/utils/request";

/**
 * sayHello
 */
export const sayHello = (username: string) => {
  return request.get(`/yst/newsys/api/v1/myTest/hello`, {
    params: { username },
  });
};
```

在demoTest/index.tsx中加入接口的调用



加入后保存即可，此时再次在页面上操作即可看到后端API接口返回的结果

🔍 菜单搜索

🏠 首页

📊 分析页

📄 示例页

☰ 我的测试

首页 我的测试 ✕

用户名

wangs

调用接口

接口返回结果

Hello, wangs